

Reimbursements Platform Blueprint

Campo	Valor
Documento	TPL-002
Nombre	Diagram Standards
Versión	1.0
Estado	Draft
Clasificación	Template
Autor	Architecture Office
Responsable	Chief Software Architect
Audiencia	Arquitectos, Líder Técnico, Líder Funcional, Desarrolladores, Revisores Técnicos, IA involucradas
Última actualización	2026-07-24

1. Propósito

Este documento define los estándares oficiales para la creación, clasificación, documentación y mantenimiento de diagramas dentro del **Reimbursements Platform Blueprint**.

Su objetivo es garantizar que las representaciones visuales utilizadas durante el proyecto sean:

- Comprensibles.
- Consistentes.
- Trazables.
- Mantenibles.
- Versionables.
- Adecuadas para su audiencia.
- Coherentes con la documentación escrita.
- Independientes de herramientas específicas siempre que sea posible.

Los diagramas deberán facilitar la comprensión del sistema.

No deberán sustituir el razonamiento arquitectónico ni convertirse en una fuente aislada de conocimiento.

2. Alcance

TPL-002 aplica a diagramas utilizados para representar:

- Negocio.
- Dominio.
- Arquitectura.
- Contextos.
- Sistemas.
- Contenedores.
- Componentes.
- Integraciones.
- Flujos.
- Procesos.
- Datos.
- Infraestructura.
- Deployment.
- Secuencias.
- Dependencias.
- Relaciones documentales.

Este documento define **cómo deberán representarse y gobernarse los diagramas**.

No define todavía:

- Bounded Contexts.
- Microservicios.
- Componentes técnicos concretos.
- Infraestructura objetivo.
- Topologías.
- APIs.
- Bases de datos.
- Estrategias de integración.
- Deployment final.

Estas decisiones pertenecerán a fases posteriores del Blueprint.

3. Relación con META-000 y TPL-000

TPL-002 se encuentra dentro de la siguiente jerarquía:

META-000
Blueprint Information Architecture

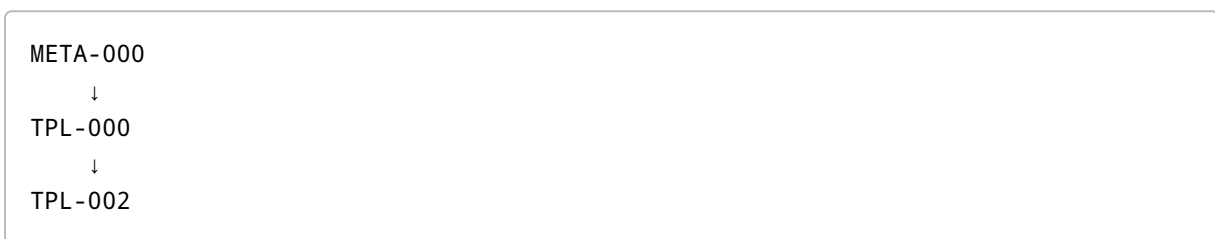


META-000 establece el gobierno del conocimiento.

TPL-000 establece el formato documental general.

TPL-002 especializa dichas reglas para representaciones visuales.

En caso de contradicción prevalecerá:



4. Principio Fundamental

Todo diagrama deberá responder:

¿Qué información intenta comunicar, a qué audiencia y con qué nivel de abstracción?

Un diagrama no deberá crearse únicamente porque una herramienta permita generarlo.

Cada representación deberá tener una intención concreta.

5. Diagram as Documentation

Los diagramas forman parte de la documentación oficial cuando:

- Están incorporados a un documento controlado.
- Son referenciados por un documento controlado.

- Poseen contexto suficiente.
- Se encuentran alineados con la versión documental correspondiente.

Un archivo gráfico aislado no constituye por sí mismo conocimiento arquitectónico oficial.

6. Principios de Diagramación

6.1 Purpose First

Antes de crear un diagrama deberá definirse su propósito.

6.2 Audience First

El nivel de detalle deberá ajustarse a la audiencia.

Un diagrama destinado al negocio no deberá requerir conocimiento profundo de infraestructura.

Un diagrama destinado a desarrolladores podrá contener mayor detalle técnico.

6.3 One Diagram, One Main Story

Cada diagrama deberá comunicar principalmente una idea.

Se evitarán representaciones que intenten mostrar simultáneamente:

- Negocio.
- Arquitectura.
- Infraestructura.
- Secuencias.
- Modelo de datos.

cuando esto reduzca su comprensión.

6.4 Explicit Abstraction Level

Todo diagrama arquitectónico deberá indicar claramente el nivel de abstracción que representa.

6.5 Consistency

Elementos equivalentes deberán representarse de forma equivalente dentro del mismo tipo de diagrama.

6.6 Traceability

Los diagramas deberán estar relacionados con los documentos que explican su significado.

6.7 Maintainability

Se favorecerán diagramas que puedan actualizarse sin reconstrucciones costosas.

6.8 Simplicity

Se incluirán únicamente elementos necesarios para comunicar el propósito del diagrama.

6.9 No Hidden Meaning

No deberá dependerse exclusivamente de:

- Colores.
- Posiciones.
- Formas ambiguas.
- Iconos sin explicación.

para comunicar información crítica.

7. Clasificación General de Diagramas

El Blueprint utilizará las siguientes categorías conceptuales:

```
Business Diagrams
Domain Diagrams
Architecture Diagrams
Interaction Diagrams
Data Diagrams
Infrastructure Diagrams
```

Operational Diagrams
Documentary Diagrams

Estas categorías representan propósitos.

No constituyen necesariamente herramientas o notaciones específicas.

8. Business Diagrams

Propósito

Representar conocimiento del negocio sin introducir decisiones técnicas.

Podrán mostrar:

- Actores.
- Procesos.
- Capacidades.
- Flujos operativos.
- Responsabilidades.
- Relaciones organizacionales.

Ejemplos futuros:

Business Capability Map
Business Process Flow
Domain Story
Actor Interaction Map

Los diagramas de negocio deberán evitar introducir prematuramente:

- Microservicios.
 - Bases de datos.
 - APIs.
 - Brokers.
 - Tecnologías específicas.
-

9. Domain Diagrams

Propósito

Representar el conocimiento obtenido durante Domain Discovery y DDD.

Podrán mostrar:

- Bounded Contexts.
- Context Maps.
- Agregados.
- Eventos de dominio.
- Comandos.
- Relaciones entre dominios.

Su contenido será definido únicamente después del correspondiente proceso de descubrimiento.

TPL-002 no autoriza su creación anticipada.

10. Architecture Diagrams

Propósito

Representar estructuras arquitectónicas y relaciones entre elementos del sistema.

Podrán existir diferentes niveles de abstracción.

La referencia conceptual preferente será el enfoque de niveles utilizado por **C4 Model**, adaptado cuando resulte útil para el proyecto.

Esto no significa que todos los diagramas deban utilizar C4 obligatoriamente.

11. Niveles Arquitectónicos

Los niveles principales podrán ser:

System Landscape
System Context

Container
Component

El uso de niveles inferiores dependerá de que exista valor real.

12. System Landscape

Propósito

Mostrar la plataforma dentro del ecosistema organizacional.

Puede incluir:

- Sistemas externos.
- Usuarios principales.
- Plataformas relacionadas.
- Integraciones de alto nivel.

No deberá mostrar implementación interna detallada.

13. System Context

Propósito

Mostrar:

- El sistema principal.
- Sus usuarios.
- Otros sistemas externos.
- Relaciones relevantes.

Debe responder:

¿Qué es el sistema y con qué actores o sistemas interactúa?

14. Container Diagram

Propósito

Mostrar las principales unidades ejecutables o de almacenamiento que constituyen un sistema.

Ejemplos conceptuales de containers podrían ser:

- Aplicación web.
- API.
- Servicio.
- Base de datos.
- Worker.
- Message Broker.

La existencia de estos elementos deberá derivarse de decisiones arquitectónicas futuras.

TPL-002 no presupone que deban existir.

15. Component Diagram

Propósito

Mostrar responsabilidades internas relevantes dentro de un container.

Este nivel deberá utilizarse únicamente cuando aporte valor.

No se crearán diagramas de componentes para cada clase o elemento de implementación.

16. Code-Level Diagrams

Diagramas detallados de:

- Clases.
- Métodos.
- Interfaces.
- Dependencias internas.

deberán utilizarse de forma excepcional.

La documentación arquitectónica no deberá intentar reflejar permanentemente toda la estructura del código mediante diagramas manuales.

Cuando dicho detalle sea necesario se favorecerá generación automatizada cuando sea viable.

17. Interaction Diagrams

Propósito

Mostrar comportamiento entre participantes.

Podrán utilizarse para representar:

- Secuencias.
 - Integraciones.
 - Flujos de mensajes.
 - Casos arquitectónicos relevantes.
-

18. Sequence Diagrams

Los diagramas de secuencia deberán utilizarse cuando el orden temporal de interacciones sea importante.

Podrán representar conceptualmente:

```
graph TD; Actor --> SystemA[System A]; SystemA --> SystemB[System B]; SystemB --> SystemC[System C];
```

Deberán evitar convertirse en una representación de cada llamada interna del sistema.

19. Diagramas de Flujo

Los diagramas de flujo podrán utilizarse para:

- Procesos.
- Decisiones.
- Estados.
- Procedimientos.

Deberán distinguir claramente:

- Inicio.
- Actividades.
- Decisiones.
- Resultado.

20. Diagramas de Estado

Podrán utilizarse cuando una entidad o proceso tenga transiciones relevantes.

Ejemplo conceptual:

```
State A
  ↓
State B
  ↓
State C
```

No deberán utilizarse para definir estados de negocio antes de realizar el descubrimiento correspondiente.

21. Data Diagrams

Propósito

Representar estructuras o relaciones de información cuando resulte necesario.

Podrán incluir:

- Modelos conceptuales.
- Modelos lógicos.
- Relaciones relevantes.
- Ownership de datos.

Los diagramas de datos no deberán utilizarse para derivar automáticamente límites de microservicios.

22. ER Diagrams

Los Entity Relationship Diagrams podrán utilizarse cuando exista necesidad de explicar un modelo relacional.

No deberán confundirse:

Modelo de datos

con:

Modelo de dominio

Una tabla no representa automáticamente una entidad de dominio.

Una agrupación de tablas no define automáticamente un Bounded Context.

23. Infrastructure Diagrams

Propósito

Representar infraestructura y deployment.

Podrán mostrar:

- Servidores.
- Redes.
- Runtime.
- Contenedores de infraestructura.
- Load balancers.
- Gateways.
- Bases de datos.
- Brokers.
- Dependencias externas.

Estos diagramas deberán crearse únicamente cuando la arquitectura de infraestructura correspondiente haya sido definida.

24. Deployment Diagrams

Los diagramas de deployment deberán explicar:

¿Dónde se ejecutan físicamente o lógicamente los elementos del sistema?

Deberán distinguir entre:

Logical Architecture

y:

Physical Deployment

para evitar mezclar responsabilidades.

25. Operational Diagrams

Podrán representar:

- Observabilidad.
- Logging.
- Métricas.
- Tracing.
- Pipelines.
- Recovery.
- Procesos operativos.

Se crearán cuando dichas estrategias sean definidas formalmente.

26. Documentary Diagrams

Los diagramas también podrán utilizarse para representar:

- Jerarquías del Blueprint.
- Dependencias documentales.
- Workflows de aprobación.
- Ciclos de vida.
- Relaciones entre artefactos.

Estos diagramas pertenecen a la arquitectura del conocimiento y podrán utilizarse durante las fases actuales.

27. Metadatos de Diagramas

Todo diagrama significativo deberá estar acompañado, directamente o mediante su documento contenedor, de:

- Identificador.
- Nombre.
- Propósito.
- Nivel de abstracción.
- Documento propietario.
- Estado.
- Última actualización.

Cuando el diagrama forme parte integral de un único documento, podrá heredar algunos de estos datos del documento.

28. Identificación

Cuando sea necesario identificar diagramas individualmente se utilizará:

DGM-<DocumentID>-<NN>

Ejemplo conceptual:

DGM-RPC-007-01
DGM-RPC-007-02

Esto permite relacionar el diagrama con su documento propietario.

DGM no constituye una nueva familia documental independiente.

Es únicamente una convención de identificación de artefactos visuales.

29. Documento Propietario

Todo diagrama oficial deberá tener un documento propietario.

Ejemplo:

DGM-RPC-007-01

↓

RPC-007 – Reference Architecture

El documento propietario contiene:

- Contexto.
- Explicación.
- Decisiones.
- Restricciones.

El diagrama representa visualmente parte de ese conocimiento.

30. Regla de Autoridad

Cuando exista una diferencia entre:

Texto aprobado

y:

Diagrama

deberá investigarse inmediatamente.

El diagrama no deberá utilizarse para modificar implícitamente una regla escrita.

La contradicción representa un defecto documental.

31. Título del Diagrama

Todo diagrama deberá tener un título significativo.

Ejemplo:

Evitar:

Diagram 1
Architecture
New Diagram
Untitled

32. Descripción

Los diagramas relevantes deberán incluir una breve explicación que responda:

- Qué representa.
- Qué no representa.
- Qué nivel de abstracción utiliza.

Ejemplo conceptual:

Este diagrama representa las interacciones externas de la plataforma y no describe su estructura interna.

33. Leyenda

Cuando un diagrama utilice:

- Colores.
- Formas.
- Líneas.
- Símbolos.
- Estilos.

con significados particulares, deberá incluir una leyenda cuando estos no sean autoexplicativos.

34. Uso del Color

El color podrá utilizarse como apoyo visual.

No deberá ser la única forma de transmitir información.

Ejemplo incorrecto:

Rojo = crítico
Verde = correcto

sin otra identificación.

Esto puede generar problemas de:

- Accesibilidad.
- Impresión.
- Exportación.
- Visualización monocromática.

Los elementos críticos deberán mantener etiquetas textuales.

35. Paleta

No se establece en TPL-002 una paleta corporativa obligatoria.

Cuando exista una identidad visual oficial podrá incorporarse mediante una versión posterior o estándar complementario.

Hasta entonces deberá favorecerse:

- Contraste adecuado.
 - Pocos colores.
 - Consistencia.
 - Legibilidad.
-

36. Formas

Las formas deberán utilizarse consistentemente dentro de un mismo diagrama.

Una misma forma no deberá representar conceptos diferentes sin explicación.

La semántica deberá derivarse principalmente de etiquetas claras.

37. Líneas y Flechas

Toda relación deberá tener dirección cuando esta sea relevante.

Se deberá distinguir cuando corresponda entre:

Dependency
Communication
Data Flow
Control Flow
Ownership

No deberán utilizarse flechas decorativas cuya semántica no sea comprensible.

38. Etiquetado de Relaciones

Cuando la relación no sea evidente deberá etiquetarse.

Ejemplo conceptual:

```
System A
  |
  | Sends event
  ▼
System B
```

Una línea sin explicación no deberá obligar al lector a adivinar su significado.

39. Protocolos y Tecnologías

En diagramas técnicos futuros podrán indicarse tecnologías o protocolos cuando estén formalmente definidos.

Ejemplo conceptual:

HTTPS
REST

No deberán introducirse tecnologías no aprobadas únicamente para completar el diagrama.

40. Boundary Representation

Los límites deberán representarse explícitamente cuando sean relevantes.

Ejemplos futuros:

- System Boundary.
- Bounded Context Boundary.
- Trust Boundary.
- Network Boundary.
- Transaction Boundary.

Cada tipo deberá identificarse para evitar interpretaciones erróneas.

41. Bounded Contexts

Cuando posteriormente se documenten Bounded Contexts:

- Sus límites deberán provenir del Domain Discovery.
- Cada contexto deberá tener nombre explícito.
- Las relaciones deberán explicarse.
- No deberán derivarse de tablas.
- No deberán confundirse automáticamente con microservicios.

TPL-002 establece únicamente la regla de representación.

No define ningún Bounded Context del proyecto.

42. Representación de Microservicios

Cuando los microservicios sean formalmente definidos:

- Deberán representar capacidades derivadas del dominio y arquitectura aprobada.
- Cada servicio deberá tener una responsabilidad comprensible.
- Las relaciones relevantes deberán mostrarse.
- La comunicación deberá indicarse cuando sea importante.

TPL-002 no autoriza la identificación actual de microservicios.

43. Bases de Datos

Cuando aparezcan bases de datos en diagramas:

- Deberá identificarse su ownership cuando sea relevante.
- No deberá inferirse automáticamente shared database.
- No deberá asumirse database-per-service sin decisión arquitectónica previa.

El diagrama deberá reflejar decisiones existentes, no crearlas.

44. Sistemas Externos

Los sistemas externos deberán diferenciarse visual o textualmente de los componentes bajo control del proyecto.

Cuando sea relevante deberá indicarse:

External System

Esto ayuda a comprender límites de responsabilidad.

45. Personas y Actores

Los actores deberán identificarse mediante roles.

Ejemplos:

Asegurado
Dictaminador
Administrador

Evitar nombres de personas reales cuando el objetivo del diagrama sea representar una responsabilidad funcional.

46. Información Sensible

Los diagramas no deberán incluir innecesariamente:

- Credenciales.
- Tokens.
- Contraseñas.
- Secretos.
- Datos personales reales.
- Direcciones privadas.
- Información sensible de infraestructura.

Cuando se necesiten ejemplos deberán utilizarse datos ficticios o abstractos.

47. Nivel de Detalle

Un diagrama deberá incluir únicamente el detalle necesario para responder su pregunta principal.

La regla será:

Mostrar lo necesario y explicar lo importante.

Más elementos no implican mejor documentación.

48. Complejidad Visual

Cuando un diagrama tenga demasiados elementos para ser comprendido razonablemente deberá dividirse.

Señales de exceso de complejidad:

- Cruces excesivos de líneas.
 - Texto ilegible.
 - Docenas de elementos sin agrupación.
 - Múltiples niveles de abstracción.
 - Necesidad constante de zoom.
 - Leyendas excesivamente grandes.
-

49. División de Diagramas

Un diagrama complejo podrá descomponerse en:

```
Overview
  ↓
Detailed View A
Detailed View B
Detailed View C
```

El overview deberá permitir comprender cómo se relacionan las vistas detalladas.

50. Consistencia entre Niveles

Cuando existan múltiples niveles:

```
System Context
  ↓
Container
  ↓
Component
```

los elementos deberán mantener coherencia.

Un container mostrado en un nivel deberá representar el mismo concepto cuando aparezca en otro.

51. Nombres Consistentes

Un concepto deberá utilizar el mismo nombre en:

- Documentos.
- Diagramas.
- ADR.
- Standards.
- Código futuro cuando corresponda.

No deberán crearse sinónimos técnicos innecesarios.

52. Ubiquitous Language

Durante y después del Domain Discovery, los diagramas de dominio deberán respetar el Ubiquitous Language aprobado.

Los diagramas no deberán introducir vocabulario alternativo que genere ambigüedad.

53. Diagramas como Código

Cuando resulte práctico se favorecerán formatos declarativos textuales para diagramas.

Ejemplos potenciales:

```
Mermaid
PlantUML
Structurizr DSL
```

La selección definitiva de herramientas deberá realizarse cuando exista necesidad concreta.

TPL-002 no impone todavía una herramienta específica.

54. Ventajas de Diagramas Declarativos

Podrán favorecer:

- Control de versiones.
- Comparación de cambios.
- Revisión mediante pull request.
- Automatización.
- Reproducibilidad.
- Integración documental.

Sin embargo, no deberán utilizarse cuando reduzcan significativamente la capacidad de expresar el diagrama necesario.

55. Herramientas Gráficas

También podrán utilizarse herramientas visuales cuando resulten más adecuadas.

El Blueprint no dependerá exclusivamente de formatos propietarios cuando pueda evitarse razonablemente.

Cuando una herramienta genere un archivo editable, deberá conservarse preferentemente la fuente además de la imagen exportada.

56. Fuente y Render

Cuando sea aplicable deberán distinguirse:

Source

y:

Rendered Artifact

Ejemplo conceptual:

architecture.mmd
architecture.svg

La fuente editable deberá considerarse el origen de mantenimiento.

57. Formatos de Exportación

Para incorporación documental se favorecerán:

SVG
PNG

Preferencia general:

SVG

cuando sea compatible, debido a su escalabilidad.

PNG podrá utilizarse cuando sea más práctico.

58. Uso de JPG

JPG debería evitarse para diagramas con:

- Texto.
- Líneas.
- Formas.
- Iconografía.

debido a pérdida de calidad por compresión.

Podrá utilizarse únicamente cuando el contenido visual lo justifique.

59. Nombres de Archivos de Diagramas

Formato recomendado:

```
<DocumentID>_<Sequence>_<Diagram_Name>.<extension>
```

Ejemplo:

```
RPC-007_01_System_Context.svg  
RPC-007_02_Container_View.svg
```

La fuente podrá utilizar el mismo nombre base:

```
RPC-007_01_System_Context.mmd  
RPC-007_01_System_Context.svg
```

60. Ubicación Física

Los diagramas asociados a un documento podrán almacenarse en una carpeta cercana al documento propietario.

Ejemplo conceptual:

```
RPC/
├── RPC-007_Reference_Architecture.md
└── assets/
    ├── RPC-007/
    │   ├── RPC-007_01_System_Context.mmd
    │   └── RPC-007_01_System_Context.svg
```

La estructura física definitiva podrá ajustarse cuando exista repositorio formal.

61. Embedding

Los documentos deberán incorporar los diagramas relevantes cerca de la sección donde se explican.

No deberá obligarse al lector a localizar manualmente un archivo sin contexto cuando pueda evitarse.

62. Texto Alternativo

Cuando la plataforma documental lo permita, las imágenes deberán incluir descripción alternativa significativa.

Esto mejora:

- Accesibilidad.
 - Búsqueda.
 - Comprensión cuando el render no está disponible.
-

63. Explicación Complementaria

Un diagrama complejo deberá ir acompañado de texto suficiente.

Regla:

El diagrama muestra; el documento explica.

No se deberá asumir que todas las implicaciones arquitectónicas son evidentes visualmente.

64. Versionado

Los diagramas heredarán normalmente la versión del documento propietario.

Ejemplo:

RPC-007 v1.2

contiene:

DGM-RPC-007-01

DGM-RPC-007-02

No será necesario asignar versiones independientes a cada diagrama salvo que exista una necesidad concreta.

65. Actualización

Cuando un cambio documental modifique una realidad representada gráficamente, los diagramas relacionados deberán revisarse.

Un diagrama desactualizado deberá considerarse un defecto documental.

66. Review de Diagramas

Durante revisión deberá comprobarse:

- Propósito claro.
 - Audiencia correcta.
 - Nivel de abstracción coherente.
 - Elementos correctamente nombrados.
 - Relaciones comprensibles.
 - Ausencia de contradicciones.
 - Legibilidad.
 - Consistencia con el documento.
 - Ausencia de información sensible.
 - Mantenibilidad.
-

67. Checklist Conceptual de Revisión

Antes de aprobar un diagrama deberá poder responderse:

1. ¿Qué intenta comunicar?
 2. ¿Quién debe entenderlo?
 3. ¿Cuál es su nivel?
 4. ¿Todos los elementos son necesarios?
 5. ¿Las relaciones son claras?
 6. ¿Los nombres coinciden con el Blueprint?
 7. ¿Introduce alguna decisión no aprobada?
 8. ¿Contradice algún documento?
 9. ¿Puede mantenerse razonablemente?
 10. ¿Necesita dividirse?
-

68. Diagramas Generados por IA

Una IA podrá:

- Proponer diagramas.
- Crear borradores.
- Convertir descripciones en representaciones visuales.
- Detectar inconsistencias.
- Generar código declarativo.
- Revisar nomenclatura.

Una IA no deberá:

- Inventar componentes.
 - Inventar microservicios.
 - Inventar relaciones.
 - Inventar tecnologías.
 - Convertir supuestos en arquitectura.
 - Presentar un diagrama generado automáticamente como arquitectura aprobada.
-

69. Validación Humana

Todo diagrama arquitectónico generado total o parcialmente mediante IA deberá ser revisado antes de formar parte de un documento Approved.

La revisión deberá validar tanto:

Corrección visual

como:

Corrección conceptual

70. Diagramas de Exploración

Durante sesiones de descubrimiento podrán existir diagramas temporales.

Ejemplos:

- Whiteboards.
- Event Storming boards.
- Bocetos.
- Mapas iniciales.

Estos artefactos podrán utilizarse para explorar conocimiento.

No se convertirán automáticamente en documentación oficial.

Para incorporarlos al Blueprint deberán:

1. Revisarse.
2. Interpretarse.
3. Normalizarse cuando corresponda.
4. Integrarse en un documento controlado.

71. Draft Diagrams

Los diagramas dentro de documentos Draft deberán considerarse igualmente provisionales.

No deberán interpretarse como arquitectura aprobada aunque visualmente parezcan completos.

72. Diagramas Históricos

Cuando un diagrama represente una arquitectura o decisión obsoleta pero tenga valor histórico podrá conservarse.

Deberá indicarse claramente:

Deprecated

o asociarse al documento histórico correspondiente.

No deberá permanecer accesible sin contexto como si representara el estado actual.

73. Diagramas As-Is

Un diagrama **As-Is** representa una situación existente.

Deberá etiquetarse explícitamente:

As-Is

74. Diagramas To-Be

Un diagrama **To-Be** representa un estado objetivo.

Deberá etiquetarse explícitamente:

To-Be

No deberán mezclarse estados As-Is y To-Be sin diferenciarlos claramente.

75. Diagramas Proposed

Cuando una representación corresponda a una propuesta aún no aprobada deberá etiquetarse:

Proposed

Esto es especialmente importante durante evaluaciones arquitectónicas.

76. Diagramas Approved

Un diagrama sólo deberá considerarse representación oficial de la arquitectura cuando forme parte de un documento Approved o tenga una relación explícita con conocimiento Approved.

77. Modelos Conceptuales vs Implementación

Los diagramas deberán diferenciar entre:

Conceptual Model
Logical Model
Physical Model

cuando exista riesgo de confusión.

78. Conceptual Model

Representa:

- Conceptos.
- Responsabilidades.
- Relaciones.

sin comprometerse con implementación.

79. Logical Model

Representa una organización estructural más concreta, pero todavía independiente de detalles físicos específicos cuando corresponda.

80. Physical Model

Representa:

- Tecnología.
- Deployment.
- Infraestructura.

- Instancias.
- Nodos.

Debe utilizarse únicamente cuando estas decisiones existan.

81. Abstracción y Fases

La profundidad de los diagramas deberá evolucionar junto al proyecto:

```
Business Discovery
  ↓
Business Views

Domain Discovery
  ↓
Domain Views

Architecture Blueprint
  ↓
Architecture Views

Technical Foundation
  ↓
Technical / Deployment Views
```

No deberán crearse vistas correspondientes a fases aún no alcanzadas para aparentar avance.

82. Source of Truth

Los diagramas son representaciones de conocimiento.

La autoridad continúa residiendo en el conjunto coherente de:

```
Blueprint
ADR
Standards
Guides
```

según las reglas de META-000.

No deberá mantenerse una arquitectura conocida únicamente porque aparece en una imagen.

83. Antipatrones de Diagramación

El Blueprint evitará:

Diagram Without Purpose

Crear diagramas sin una pregunta concreta.

Everything Diagram

Representar toda la plataforma en una única imagen.

Architecture by Icons

Utilizar únicamente logos de tecnologías sin expresar responsabilidades.

Boxes and Lines Without Meaning

Crear cajas y flechas sin semántica clara.

Color Dependency

Comunicar información crítica exclusivamente mediante color.

Mixed Abstraction

Mostrar simultáneamente conceptos de negocio, clases de código, servidores y tablas sin separación.

Premature Architecture

Dibujar componentes técnicos antes de entender el dominio.

Diagram as Decision

Utilizar un diagrama para introducir una decisión que nunca fue analizada ni aprobada.

Stale Diagram

Mantener representaciones que ya no corresponden con la documentación vigente.

Tool-Driven Architecture

Diseñar la arquitectura según las capacidades visuales de una herramienta.

Decorative Complexity

Agregar iconos, sombras, capas o elementos visuales que no mejoran comprensión.

84. Plantilla de Registro de Diagrama

Cuando un diagrama requiera descripción formal podrá utilizarse:

```
## <Diagram Name>

| Campo | Valor |
|---|---|
| Identificador | DGM-XXX-000-01 |
| Tipo | System Context / Sequence / etc. |
| Estado | Draft |
| Nivel | Conceptual / Logical / Physical |
| Documento propietario | XXX-000 |
| Última actualización | YYYY-MM-DD |

### Propósito

Describe qué pregunta responde el diagrama.

### Audiencia

Describe los lectores principales.
```

Alcance

Describe qué representa.

Fuera de alcance

Describe qué no debe inferirse del diagrama.

Diagrama

[Representación visual]

Notas

Describe decisiones, convenciones o aclaraciones necesarias.

Esta estructura será opcional cuando el contexto ya esté claramente definido por la sección documental contenedora.

85. Representación Textual Simple

Para relaciones sencillas podrá utilizarse un diagrama textual directamente dentro de Markdown.

Ejemplo:

META-000

↓

TPL-000

↓

TPL-001

↓

TPL-002

No será necesario generar una imagen para representar estructuras simples.

86. Criterio de Selección

Antes de elegir el formato deberá aplicarse:

¿Texto simple comunica correctamente?

— Sí → Utilizar texto.

— No



¿Diagrama declarativo es suficiente?

— Sí → Utilizarlo.

— No



Utilizar herramienta visual adecuada.

La herramienta más compleja no deberá seleccionarse automáticamente.

87. Criterios de Calidad

Un diagrama de calidad deberá ser:

- Correcto.
- Intencional.
- Legible.
- Consistente.
- Trazable.
- Simple.
- Mantenible.
- Accesible.
- Coherente con su nivel.
- Coherente con el Blueprint.

88. Criterios de Aceptación de TPL-002

TPL-002 podrá pasar a Approved cuando:

- La clasificación de diagramas sea aceptada.
- Los niveles arquitectónicos sean validados.
- La relación con C4 como referencia conceptual sea aceptada.
- Las reglas de identificación sean aprobadas.
- El modelo de documento propietario sea aceptado.
- Las reglas de abstracción sean aprobadas.
- Las convenciones de nombres sean validadas.

- Las reglas para As-Is, To-Be y Proposed sean aceptadas.
- Las reglas de versionado y mantenimiento sean aprobadas.
- Los antipatrones sean aceptados.
- La política de herramientas permanezca suficientemente neutral.
- No existan contradicciones conocidas con META-000 o TPL-000.

Hasta entonces permanecerá:

Draft

89. Restricciones de Fase

Mientras TPL-002 permanezca en Draft o In Review:

- No se utilizará como estándar documental aprobado.
- No se iniciará TPL-003.
- No se iniciará RPC-000.
- No se iniciará Business Discovery.
- No se iniciará Domain Discovery.
- No se identificarán Bounded Contexts.
- No se diseñarán microservicios.
- No se diseñará infraestructura.
- No se generará arquitectura técnica.
- No se generará código.

Los ejemplos utilizados en este documento son exclusivamente ilustrativos.

90. Próximo Gate

El gate actual será:

TPL-002
Diagram Standards

Draft

↓

Review

↓

Approved

Únicamente después de su aprobación podrá iniciarse:

91. Relación con otros documentos

Documentos padre

- META-000 — Blueprint Information Architecture
- TPL-000 — Document Template Standard

Documento especializado relacionado

- TPL-001 — ADR Template

Documentos fundacionales relacionados

- BOOT-003 — Architecture Vision
- BOOT-004 — AI Collaboration Model
- BOOT-006 — Project Roadmap

Documento siguiente condicionado

- TPL-003 — Prompt Template

Documentos futuros relacionados

- RPC-000 — Blueprint Manifest
 - RPC-003 — Architecture Governance
 - RPC-004 — Business Discovery Guide
 - RPC-005 — Domain Modeling Guide
 - RPC-007 — Reference Architecture
-

92. Principio Final

Los diagramas del Blueprint seguirán el principio:

Un buen diagrama reduce la cantidad de conocimiento que debe explicarse, pero nunca elimina la necesidad de comprenderlo.

La representación visual deberá servir a la arquitectura.

La arquitectura nunca deberá deformarse para servir a la representación visual.

93. Historial de Cambios

Versión	Fecha	Descripción
1.0	2026-07-24	Creación inicial de TPL-002 — Diagram Standards. Documento generado en estado Draft para revisión.

Estado

Draft — Pending Review

No se autoriza el inicio de TPL-003 hasta completar la revisión y aprobación de TPL-002.